

IHK Abschlussprüfung Teil 2 – Sommer 2025	 Industrie- und Handelskammer zu Rostock
IT- Berufe Deckblatt für die Dokumentation zur Betrieblichen Projektarbeit	Ausbildungsberuf: Bitte Beruf auswählen:

Titel der Betrieblichen Projektarbeit: Simulationsprojekt: „Murmelsimulation – Wie beeinflussen physikalische Parameter die Murmel?“	
Prüfungsteilnehmer/in Vor- und Familienname: Benjamin Seidel Anschrift des Prüfungsteilnehmers: OT: Klicken Sie hier, um den Ortsteil einzugeben. Straße: Friedrich-Wolf-Straße 27 PLZ Ort 18435 Telefon: -491729289623 E-Mail: benjamin.159@web.de	Ausbildungs-/ Praktikumsunternehmen: Verantwortliche/r für die Durchführung des betrieblichen Auftrages: Klicken Sie hier, um Vor- und Zunamen des Ansprechpartners im Unternehmen einzugeben. Anschrift des Unternehmens: OT: Klicken Sie hier, um den Ortsteil einzugeben. Straße: Klicken Sie hier, um die Straße einzugeben. PLZ Ort Klicken Sie hier, um Postleitzahl und Ort einzugeben. Telefon: Klicken Sie hier, um die Telefonnr. des Ansprechpartners einzugeben. E-Mail: Klicken Sie hier, um die E-Mail-Adresse des Ansprechpartners einzugeben.

_____	Klicken Sie hier, um ein Datum einzugeben.	_____
Ort	Datum	Unterschrift Antragsteller/-in (Prüfling)

_____	Klicken Sie hier, um ein Datum einzugeben.	_____
Ort	Datum	Stempel/ Unterschrift Verantwortliche/r für den betrieblichen Auftrag

Inhaltsverzeichnis

Abkürzungsverzeichnis.....	1
1. Einleitung.....	2
1.1. Ausgangssituation und Problemstellung.....	2
1.2. Projektziele.....	2
1.3. Rahmenbedingungen und Abgrenzungen.....	2
1.4. Anforderungen an die Anwendung.....	3
1.4.1 Funktionale Anforderungen.....	3
1.4.2 Nicht-funktionale Anforderungen.....	3
2. Projektplanung.....	4
2.1. Vorgehensmodell.....	4
2.2. Ressourcen- und Ablaufplanung.....	4
2.2.1. Zeitplanung.....	4
2.2.2. Kostenplanung.....	5
2.3. Risikoanalyse.....	5
3. Technische Umsetzung.....	6
3.1. Architektur und Design.....	6
3.1.1. Architektur-Konzept (Hybrid / Actor-Component).....	6
3.1.2. Datenauswertung.....	6
3.2. Auswahl der Technologien.....	7
3.2.1. Engine.....	7
3.2.2. Integrierte Entwicklungsumgebung.....	7
3.2.3. Modellierungssoftware.....	8
3.3. Implementierung.....	8
3.3.1. Kernkomponenten.....	8
3.3.2. Zufallselemente und Verteilung.....	9
3.4. Benutzeroberfläche.....	10
3.4.1. UI-Konzept und Benutzbarkeit.....	10
3.4.2. Elemente und Animation.....	10
4. Qualitätssicherung.....	11
4.1. Testkonzept.....	11
4.2. Testdurchführung.....	11
4.3. Soll-Ist-Vergleich.....	12
5. Zusammenfassung und Ausblick.....	13
5.1. Fazit.....	13
5.2. Ausblick.....	13

Literaturverzeichnis.....	14
Anhang.....	i
Tabellen:.....	i
Bilder:.....	iv



Abkürzungsverzeichnis

Abkürzung	Bedeutung
BP	Blueprint (Visual Scripting in Unreal Engine)
CCD	Continuous Collision Detection
GUI	Graphical User Interface (Grafische Benutzeroberfläche)
HUD	Heads-Up Display
IDE	Integrated Development Environment (Integrierte Entwicklungsumgebung)
IHK	Industrie- und Handelskammer
ISO	International Organization for Standardization
IT	Informationstechnik
PC	Personal Computer
UE5	Unreal Engine 5
UI	User Interface (Benutzerschnittstelle)
UMG	Unreal Motion Graphics (UI-Framework der Unreal Engine)
UX	User Experience (Benutzererfahrung)

Tabelle 1: Abkürzungsverzeichnis



1. Einleitung

1.1. Ausgangssituation und Problemstellung

Das hier dokumentierte Projekt „Gravity Rollers“ ist die vorgegebene Projektarbeit für das dritte Ausbildungsjahr.

Das Kernziel der Anwendung ist die Entwicklung einer physikbasierten, zeitabhängigen Simulation. Dem Anwender soll ein Werkzeug bereitgestellt werden, um die Auswirkungen von konfigurierbaren physikalischen Parametern und definierten Zufallsfaktoren auf einer Murmelbahn visuell zu analysieren und auszuwerten.

Die Realisierung erfolgt als Windows-Anwendung mit grafischer Benutzeroberfläche. Eine wesentliche technische Herausforderung ergibt sich aus der Anforderung, die Lauffähigkeit auf den Schulrechnern als Zielsystem zu gewährleisten.

1.2. Projektziele

Die primären Projektziele umfassen die Entwicklung einer intuitiven und benutzerfreundlichen grafischen Benutzeroberfläche.

Ebenso zentral ist die nachvollziehbare visuelle Darstellung der Simulation. Der Anwender muss die Auswirkungen der individuell konfigurierten physikalischen Parameter sowie der implementierten Zufallsereignisse auf das Endergebnis analysieren und auswerten können.

Die Abgabe der Anwendung erfolgt als einzelne, auf Windows lauffähige, ausführbare Datei.

1.3. Rahmenbedingungen und Abgrenzungen

Das Projekt unterliegt zeitlichen, technischen und organisatorischen Rahmenbedingungen, die vorgegeben sind.

Zeitlicher und Organisatorischer Rahmen: Das Projekt ist im Rahmen der schulischen Projektarbeit durchzuführen. Der fixe Abgabetermin für die Dokumentation und das fertige Softwareprodukt ist der 06.02.2026.

Technische Rahmenbedingungen: Als Zielplattform ist eine ausführbare 64-Bit-Windows-Anwendung (.exe) definiert. Eine zwingende Anforderung ist die performante Lauffähigkeit der Simulation auf der Standard-Hardware der Schulrechner.

Inhaltliche Abgrenzung: Um den Projektumfang im vorgegebenen Zeitrahmen realisieren zu können, werden folgende Funktionalitäten explizit ausgeschlossen:

- Es wird kein Speichersystem für rennübergreifende Statistiken implementiert.



- Ein Level-Editor zur Erstellung eigener Strecken ist nicht Bestandteil des Projekts.
- Die Anwendung wird als reine Single-User-Simulation konzipiert.
- Der Fokus liegt auf der Konfiguration und Auswertung eines einzelnen Rennlaufs.
- Die Anwendung wird keine Umgebungsgeräusche oder Musik beinhalten.

1.4. Anforderungen an die Anwendung

1.4.1 Funktionale Anforderungen

Die funktionalen Anforderungen definieren den konkreten Leistungsumfang der Anwendung. Sie beschreiben die spezifischen Aufgaben und Funktionen, die das System erfüllen muss.

- **Zeitabhängige Simulation:** Die Kernfunktionalität muss eine Physik-Simulation sein, deren Ablauf und Ergebnis von der verstrichenen Zeit abhängen.
- **Konfigurierbarkeit:** Dem Anwender müssen mindestens sieben physikalische Eingabeparameter zur Verfügung stehen..
- **Zufallsfaktoren:** Es sind mindestens drei unterschiedliche Zufallsverteilungen zu implementieren, um unvorhersehbare Einflüsse auf die Simulation abzubilden.
- **Geschwindigkeitssteuerung:** Die Abspielgeschwindigkeit der Simulation muss durch den Benutzer in mindestens drei Stufen regelbar sein.
- **Visualisierung & Auswertung:** Der Simulationsverlauf ist grafisch darzustellen. Nach Abschluss eines Durchlaufs müssen die Ergebnisse visuell aufbereitet werden.
- **Animation:** Es ist mindestens eine thematisch passende Animation zu integrieren.
- **Bereitstellung:** Die Anwendung muss als eigenständige, ausführbare Datei kompiliert werden, die auf der definierten Zielhardware lauffähig ist.

1.4.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen legen die Qualitätskriterien fest, unter denen die Anwendung betrieben und weiterentwickelt wird. Sie definieren die Rahmenbedingungen für die technische Umsetzung, die Qualität der Benutzerinteraktion sowie die Wartbarkeit.

- **Benutzbarkeit:** Die Gestaltung der grafischen Benutzeroberfläche und die Interaktionsführung müssen die Prinzipien der ISO-Norm 9241-110 berücksichtigen.
- **Code-Qualität:** Der Quellcode ist nach den Kriterien von „Clean Code“ zu strukturieren, um Lesbarkeit, Wartbarkeit und Erweiterbarkeit sicherzustellen.
- **User Experience:** Durch den gezielten Einsatz von Feedback-Elementen soll eine positive Nutzererfahrung gewährleistet werden.



2. Projektplanung

2.1. Vorgehensmodell

Die Realisierung des Projekts erfolgt anhand des Wasserfallmodells. Diese Entscheidung für ein lineares, sequenzielles Vorgehen ist durch die Rahmenbedingungen begründet: Der Abgabetermin ist fix auf den 06.02.2026 terminiert und der Funktionsumfang wurde im Projektauftrag als unveränderliche Anforderung (Fixed Scope) definiert.

Das Modell sieht vor, dass die Entwicklung in aufeinanderfolgenden, logisch abgegrenzten Phasen erfolgt, wobei jede Phase erst nach Abschluss der vorhergehenden begonnen wird:

1. **Analysephase:** Detaillierung der Anforderungen.
2. **Entwurfsphase:** Planung der Architektur und des UI-Designs.
3. **Implementierungsphase:** Programmierung der Simulationslogik sowie der GUI.
4. **Qualitätssicherung:** Durchführung von Funktionaltests und Soll-Ist-Vergleich.
5. **Dokumentation:** Erstellung der Projektdokumentation.

Durch diese strikte Phasenorientierung wird eine exakte Überwachung des Projektfortschritts gewährleistet (siehe Zeitplanung in Kap. 2.2.1). Ein entscheidender Vorteil dieses Ansatzes liegt in der Prävention von Scope Creep (ungeplante Umfangserweiterung), da die Anforderungen bereits vor Beginn der Implementierung finalisiert werden. Agile Methoden wie Scrum wurden bewusst verworfen, da sie ihre Stärken primär in iterativen Feedback-Schleifen ausspielen. Das inhärente Risiko des Wasserfallmodells, die späte Identifikation von Fehlern, wird durch eine intensivierete Analyse- und Designphase proaktiv minimiert.

2.2. Ressourcen- und Ablaufplanung

2.2.1. Zeitplanung

Für das Projekt steht ein Gesamtzeitraum von 23 Wochen (06.09.2025 bis 06.02.2026) zur Verfügung. Um die Vergleichbarkeit mit betrieblichen Projekten zu gewährleisten, wird ein Netto-Arbeitsaufwand von 160 Stunden veranschlagt. Dies entspricht einer durchschnittlichen Arbeitszeit von 7 Stunden pro Woche.

Die Phasen verteilen sich wie folgt auf den Projektzeitraum:

Verdeutlicht in Schaubild 1: Gantt-Diagramm zur Zeitplanung



2.2.2. Kostenplanung

Die Kostenplanung betrachtet die theoretischen Entwicklungskosten des Projekts. Da es sich um ein internes Ausbildungsprojekt handelt, werden keine Gewinne oder Umsatzsteuern ausgewiesen. In der Kostenplanung werden die Personalkosten, anhand des Stundesatzes und die Sachmittelkosten berücksichtigt. Alle genutzten Assets sind auch noch in dieser Tabelle zu finden mit ihrer Quelle Tabelle 4: Alle genutzten vorgefertigten Assets.

Gesamtkosten:

Position	Berechnung	Betrag
Personalkosten	160 Std. $\times$ 5,00 €	800,00 €
Software-Lizenzen	Open-Source: Blender, UE5 Bildungslizenz: Rider	0,00 €
Hardware-Nutzung	Bestand	0,00 €
Plugin	MiniGraph – God HUD	11,33 €
Assets	Old Brick House, Laptop Low-poly, Simple Standing Fan, 5“ Tri-Blade Propeller, Animalia - German Shepherd, Zenitha Classic Schriftart, TT Jenevers Schriftart	0,00 €
Gesamtsumme		811,33 €

Tabelle 2: Kostenplanung

2.3. Risikoanalyse

Zur Sicherstellung des Projekterfolgs wurde im Vorfeld eine detaillierte Betrachtung möglicher Störfaktoren durchgeführt. Ziel dieses Risikomanagements ist es, technische und organisatorische Unsicherheiten frühzeitig zu erkennen und durch definierte Gegenmaßnahmen handhabbar zu machen. Die Bewertung der einzelnen Risiken erfolgt dabei anhand ihrer Eintrittswahrscheinlichkeit sowie der potenziellen Auswirkung auf die Projektziele.

Zusammengetragen in Tabelle 3: Risikoauflistung



3. Technische Umsetzung

3.1. Architektur und Design

3.1.1. Architektur-Konzept (Hybrid / Actor-Component)

Das Projekt folgt dem Actor-Component-Modell der Unreal Engine, erweitert um eine hybride Schichtenarchitektur aus C++ und Blueprints. Dieses Entwurfsmuster wurde gewählt, um eine klare Trennung der Verantwortlichkeiten zu gewährleisten und gleichzeitig die Performanz der Simulation mit der Flexibilität der visuellen Gestaltung zu vereinen.

Die Architektur gliedert sich in drei logisch getrennte Bereiche:

- **Logik-Ebene:** Es kapselt die gesamte Simulationslogik, die physikalischen Berechnungen und die Zustandsdaten. Durch die Implementierung in C++ wird die notwendige Rechenleistung für die Physik-Engine sichergestellt.
- **Visuelle Ebene:** Die Darstellung und das Feedback an den Nutzer erfolgen über abgeleitete Blueprint-Klassen und das UMG-Framework.
- **Steuerungs-Ebene:** Der APlayerController dient als Schnittstelle zwischen Eingabegerät und Logik.

Betrachtung von Alternativen: Als Alternative wurde eine reine Blueprint-Architektur (Visual Scripting) in Betracht gezogen. Diese hätte die Einstiegshürde gesenkt und eine schnellere Prototypen-Entwicklung ermöglicht. Allerdings wurde dieser Ansatz verworfen, da komplexe mathematische Berechnungen in Blueprints zu einer schlechteren Wartbarkeit und signifikanten Performance-Einbußen führen würden. Gerade im Hinblick auf die begrenzten Ressourcen der Schulrechner bietet der gewählte Hybrid-Ansatz den entscheidenden Vorteil. Zudem erleichtert die Kapselung der Physik in C++ das spätere Debugging und Testen der Kernfunktionen.

3.1.2. Datenauswertung

Die Analyse der Simulationsdaten erfolgt visuell über das Plugin MiniGraph. Dieses rendert performante Graphen auf dem virtuellen In-Game-Laptop.

Die Darstellung gliedert sich in drei Bereiche:

- **Line Graph:** Stellt den Geschwindigkeitsverlauf über die Zeit dar.
- **Histogramm:** Ein Säulendiagramm zeigt die Häufigkeitsverteilung der Geschwindigkeitswerte.



- Tabelle: Zeigt die wichtigsten Geschwindigkeiten an und die eingestellten physikalischen Parameter.

Technische Umsetzung: Die Messwerte werden während der Simulation frame-genau in der AMarble-Klasse erhoben (GetVelocity), vom DataTrackerPlugin gepuffert und nach Rennende als strukturierter Datensatz an das UI-System zur Darstellung übermittelt.

3.2. Auswahl der Technologien

3.2.1. Engine

Da die realistische Darstellung physikalischer Kräfte die Kernanforderung des Projekts darstellt, lag der Fokus der Bewertung auf der Simulationsgenauigkeit.

Kriterien für die Nutzwertanalyse:

- **Physik-Engine (Gewichtung: 4):** Für eine physikalische Simulation ist die Qualität und Stabilität der Physik-Engine das wichtigste Kriterium.
- **GUI-Framework (Gewichtung: 3):** Die Anforderung einer ISO-konformen Benutzeroberfläche erfordert leistungsfähige, integrierte UI-Werkzeuge.
- **Programmiersprache (Gewichtung: 2):** Die Unterstützung einer performanten Sprache ist wichtig für die Optimierung.
- **Dokumentation & Community (Gewichtung: 2):** Wichtig für die Problemlösung.
- **Einarbeitungszeit (Gewichtung: 1):** Da die technische Eignung im Vordergrund steht, wird der Einarbeitungsaufwand als weniger kritisch eingestuft.

Durchführung in Tabelle 5: Nutzwertanalyse Engine.

Begründung der Entscheidung: Die Wahl fiel auf die Unreal Engine 5. Ausschlaggebend war der Hybrid-Ansatz. Unity und Godot boten insbesondere bei der integrierten Physik-Performance Nachteile im Vergleich zur Chaos Engine.

3.2.2. Integrierte Entwicklungsumgebung

Da das Projekt auf einem C++ Hybrid-Ansatz basiert, ist eine leistungsfähige IDE für das Schreiben, Refactoring und Debuggen des Codes essenziell.

Kriterien für die Nutzwertanalyse:

- **Unreal-Integration (Gewichtung: 4):** Die IDE muss das komplexe Unreal-Build-System und spezifische C++ Makros verstehen.
- **Performance (Gewichtung: 3):** Schnelles Indizieren und ein zuverlässiger Debugger sind für flüssiges Arbeiten nötig.
- **Kosten (Gewichtung: 2):** Die Wirtschaftlichkeit muss betrachtet werden.



- **Plugins (Gewichtung: 1):** Die Verfügbarkeit von nützlichen Zusatztools.

Durchführung in Tabelle 6: Nutzwertanalyse IDE.

Begründung der Entscheidung: Trotz der grundsätzlichen Kostenpflicht fiel die Wahl auf JetBrains Rider. Der entscheidende technische Vorteil ist die tiefe Unreal-Integration. Der wirtschaftliche Nachteil entfällt durch eine kostenlose Bildungslizenz. Visual Studio Code wurde aufgrund der aufwendigen manuellen Konfiguration verworfen.

3.2.3. Modellierungssoftware

Für die Erstellung und Bearbeitung der 3D-Assets (Murmeln, Streckenteile) war die Wahl einer geeigneten 3D-Software notwendig.

Kriterien für die Nutzwertanalyse:

- **Kompatibilität (Gewichtung: 4):** Der Export zur gewählten Engine muss reibungslos funktionieren.
- **Lizenzkosten (Gewichtung: 4):** Da für diesen Bereich kein Budget vorgesehen ist, sind kostenlose oder Open-Source-Lösungen stark zu bevorzugen.
- **Funktionsumfang (Gewichtung: 2):** Grundlegende Funktionen müssen vorhanden sein.

Durchführung in Tabelle 7: Nutzwertanalyse Modellierungssoftware .

Begründung der Entscheidung: Die Entscheidung fiel auf Blender. Neben dem ausreichenden Funktionsumfang war die Lizenzfreiheit das entscheidende Argument. Zudem bietet Blender inzwischen eine exzellente Integration in den Unreal-Workflow, was Konvertierungsprobleme minimiert.

3.3. Implementierung

3.3.1. Kernkomponenten

Die Anwendungslogik verteilt sich auf fünf spezialisierte C++-Klassen.

AMarble: Diese Klasse repräsentiert den physikalischen Körper der Simulation und übernimmt die Geschwindigkeitsmessung.

AMarbleWorkbench: Dient als "Staging Area" vor der eigentlichen Simulation und macht die physikalischen Parameter der AMarble konfigurierbar.

ARaceTrack: Verantwortlich für das Spawnen der Akteure und die Validierung der Rennbedingungen. Aktiviert die Wirkung der Zufallsfaktoren.

AMarbleGameMode: Die zentrale Steuerinstanz, die den Status der Simulation überwacht.

AMarblePlayerController: Verbindet die Benutzereingaben mit der Logik.



Alle Klassen werden in dem Schaubild 2: Klassendiagramm (nur die Beziehungen) dargestellt. In Schaubild 3: Klassendiagramm (Teil 1 mit Attributen und Funktionen) und Schaubild 4: Klassendiagramm (Teil 2 mit Attributen und Funktionen) sind die Attribute und Funktionen der Klassen enthalten, für die Übersichtlichkeit wurden die UI-Klassen die mit dem Blueprintsystem der Unreal Engine erstellt wurden vereinfacht dargestellt und auf die Vererbung der Standard-Unreal-Engine-Klassen (AActor und UUserWidget) verzichtet.

Interaktion der Komponenten: Der Ablauf folgt einem strikten Schema:

1. Konfiguration: Der Nutzer stellt Werte in der UI ein, der PlayerController aktualisiert die MarbleWorkbench.
2. Initialisierung: Beim Start ruft RaceTrack die Daten von der Workbench ab und spawnnt die echten Physik-Murmeln.
3. Simulation: Der GameMode startet die Zeit. Marble berechnet Physik und sendet Telemetrie.
4. Abschluss: Trigger in RaceTrack melden das Ende an Marble und GameMode. Der GameMode beendet die Session und der PlayerController wechselt in die Analyse-Ansicht.

3.3.2. Zufallselemente und Verteilung

Zur Simulation organischer Umwelteinflüsse und zur Steigerung der Varianz kommen drei stochastische Verteilungsmodelle zum Einsatz.

- **Exponentialverteilung (ADog):** Steuert die Zeitabstände der „Stoß“-Events. Durch die Implementierung als Poisson-Prozess entstehen unvorhersehbare Intervalle, was die Spannung im Vergleich zu festen Timern signifikant erhöht. Um gesetzt durch folgende Formel: $t_{next} = t_{min} + (-\ln(U) * \mu)$
- **Normalverteilung (AFan):** Die Windkraft wird mittels der Box-Muller-Transformation berechnet. Dies simuliert realistische Luftströmungen, die dynamisch um einen Mittelwert (MeanWindStrength) schwanken, anstatt eine statische Kraft auszuüben. Die Berechnung erfolgt mit: $Z_0 = \sqrt{-2 * \ln(U_1)} * \cos(2 * \pi * U_2)$ und $Windkraft = \mu + (Z_0 * \sigma)$
- **Gleichverteilung (AObstacleGenerator):** Die Platzierung und Typ-Auswahl der Hindernisse erfolgt uniform (FMath::RandRange). Dies verhindert Clusterbildung innerhalb der SpawnArea und garantiert bei jedem Durchlauf ein faires, aber abwechslungsreiches Streckenlayout.



3.4. Benutzeroberfläche

3.4.1. UI-Konzept und Benutzbarkeit

Die Gestaltung der Benutzeroberfläche orientiert sich an den Grundsätzen der Dialoggestaltung nach ISO 9241-110.

- **Aufgabenangemessenheit:** Das Main-HUD reduziert sich auf essentielle Funktionen. Rennspezifische Elemente werden erst bei Rennstart eingeblendet.
- **Selbstbeschreibungsfähigkeit:** Die Konfiguration der Physikwerte erfolgt direkt am Objekt. Ein Klick auf eine Murmel öffnet das Konfigurationsmenü diese werden auf alle UI-Elemente übertragen.
- **Steuerbarkeit:** Über das HUD kann die Simulation jederzeit pausiert, beschleunigt (Time Dilation) oder zurückgesetzt werden.
- **Fehlertoleranz:** Sollte sich eine Simulation in einen unerwünschten Zustand bewegen, ermöglicht die Reset-Funktion eine Wiederherstellung.

Layout und Zustände: Die Oberfläche gliedert sich dynamisch in vier Phasen:

1. Setup: Fokus auf der Interaktion mit 3D-Objekten (Murmel-Konfiguration).
2. Simulation: Einblendung der Zeit und Geschwindigkeitssteuerung im HUD.
3. Analyse: Automatischer Kamerawechsel zur Datenauswertung.
4. Einstellungen: Ein Optionsmenü das jeder Zeit aufrufbar ist.

Dies wird im Schaubild 5: Use-Case-Diagramm der Simulation auch deutlich.

3.4.2. Elemente und Animation

Die visuelle Darstellung der Murmelbewegung erfolgt nicht durch vorgefertigte Keyframes, sondern als Echtzeit-Simulation der Chaos Physics Engine.

- **Kamerasteuerung:** Übergänge erfolgen zwischen den Ansichten nicht als harte Schnitte, sondern als interpolierte Kamerafahrten (SetViewTargetWithBlend).
- **Interaktive Aktoren:** Umgebungs-Objekte visualisieren ihren Status direkt. Der ADog-Actor nutzt eine State Machine für Zustandsanimationen (Idle, Lying, Jump). Der Ventilator zeigt seinen Aktivierungsstatus (bFanOn) durch Rotation an.
- **Eingabebeschränkung:** Zur Fehlervermeidung werden Slider, Spinboxes und Checkboxes eingesetzt. Diese limitieren Eingaben technisch auf sinnvolle Wertebereiche (Clamping).



4. Qualitätssicherung

4.1. Testkonzept

Die Qualitätssicherung verfolgte das Ziel, die funktionale Korrektheit der physikalischen Simulation sowie die Robustheit der Benutzeroberfläche sicherzustellen. Aufgrund der starken visuellen und interaktiven Komponente des Projekts wurde primär auf manuelle Systemtests (Black-Box-Verfahren) gesetzt.

Das Testkonzept gliedert sich in zwei Phasen:

1. **Entwicklertests (Laufend):** Während der Implementierung wurden atomare Funktionen (z. B. das Spawnen einer Murmel) direkt im Editor überprüft.
2. **Systemtest (Final):** Nach Fertigstellung des Release-Builds („Packaged Game“) wurde ein umfassender Testkatalog abgearbeitet, um sicherzustellen, dass die Anwendung auch außerhalb der Entwicklungsumgebung fehlerfrei läuft.

Testumgebung: Die Tests wurden auf einem Standard-Windows-PC (Zielplattform) durchgeführt, um reale Nutzungsbedingungen zu simulieren. Eingabegeräte waren Maus und Tastatur.

Testkategorien: Der erstellte Testplan (siehe Anhang für das vollständige Protokoll) deckt folgende Bereiche ab:

- **UI-Interaktion:** Reaktion von Buttons, Slidern und Menüs (z. B. Tooltips, Grenzwerte bei Spinboxen).
- **Spiellogik:** Korrekter Ablauf des Rennens (Start, Pause, Reset, Zieleinlauf).
- **Physik & Zufall:** Plausibilitätsprüfung der Zufallsgeneratoren und physikalischen Kräfte (Ventilator, Hund).
- **Performance:** Langzeittest (1 Stunde Laufzeit) zur Prüfung auf Speicherlecks oder Framerate-Einbrüche.

4.2. Testdurchführung

Die Durchführung erfolgte anhand eines Testprotokolls mit insgesamt 50 Testfällen. Zusätzlich wurde die Anwendung Testpersonen aus dem Bekanntenkreis zur freien Nutzung überlassen, um die intuitive Bedienbarkeit (Usability) zu validieren. Die Einzelheiten sind im Tabelle 8: Testprotokoll zu finden.



Zusammenfassung der Ergebnisse: Von den 50 definierten Testfällen konnten 49 erfolgreich („Bestanden“) abgeschlossen werden. Die Kernfunktionen der Simulation arbeiten stabil und erwartungskonform.

- **Positive Ergebnisse:** Kritische Funktionen wie die Datenübertragung von der Workbench zur Rennstrecke, die Zeitmessung und die grafische Auswertung funktionierten fehlerfrei. Auch der Stresstest (50 Rennen in einer Stunde) zeigte keine Performance-Probleme.
- **Abweichungen:** Testfall Nr. 25 („Sprache umschalten“) schlug im gepackten Build fehl. Während die Lokalisierung im Editor funktionierte, wechselte die Sprache in der ausführbaren Datei (.exe) nicht korrekt auf Englisch. Beim Durchlaufen der Simulation ist noch aufgefallen, dass das 3D-Modell der Rennstrecke besonders in den Kurven Mängel aufweist.
- **Maßnahme:** Da die Mehrsprachigkeit als „Kann-Anforderung“ (Nice-to-have) definiert war, aber der Zeitplan es erlaubte, wurde der Fehler beim nächsten Package behoben. Aus Zeitgründen und der fehlenden Modellierungserfahrung wird auf die Erstellung einer neuen Rennstrecke verzichtet.

4.3. Soll-Ist-Vergleich

Der abschließende Vergleich des entwickelten Softwareprodukts mit den Anforderungen des Projektauftrags zeigt, dass alle funktionalen und nicht-funktionalen Ziele erreicht wurden.

Insbesondere die Kernanforderung, eine zeitabhängige Simulation mit mindestens drei unterschiedlichen mathematischen Zufallsverteilungen (Normal-, Gleich- und Exponentialverteilung) zu implementieren, wurde vollständig umgesetzt. Die Anwendung liegt als ausführbare Windows-Datei vor und ermöglicht die Steuerung über die geforderte Anzahl an Eingabeparametern. Auch die qualitativen Vorgaben hinsichtlich Clean Code und der ISO 9241-110 wurden im Entwicklungsprozess berücksichtigt. Eine detaillierte Gegenüberstellung aller Einzelanforderungen und deren Erfüllungsgrad befindet sich im Tabelle 9: Soll-Ist-Vergleich.



5. Zusammenfassung und Ausblick

5.1. Fazit

Das Projekt konnte erfolgreich abgeschlossen werden, alle im Soll-Konzept definierten Kernanforderungen, insbesondere die Implementierung der mathematischen Zufallsverteilungen, wurden funktionstüchtig umgesetzt.

Rückblickend war die Arbeit mit der Unreal Engine 5 besonders lehrreich. Ein zentraler Erkenntnisgewinn lag in der hybriden Entwicklung: Die Synergie zwischen C++ (für die Backend-Logik und mathematische Berechnungen) und dem Blueprint-System (für visuelles Scripting und UI) hat sich als extrem effizient erwiesen.

Eine wichtige „Lesson Learned“ betrifft den Bereich 3D-Modellierung. Während die Programmierung im Zeitplan lag, wurde der Aufwand für das Erstellen eigener 3D-Assets (Strecke, Hindernisse) unterschätzt. Es wurde deutlich, dass 3D-Artistik ein eigenes Fachgebiet ist, das viel Einarbeitungszeit erfordert. Das Ergebnis ist funktional, zeigt aber an einigen Stellen der Strecke optisches Verbesserungspotenzial. Für zukünftige Projekte würde ich hier entweder mehr Zeit einplanen oder auf vorgefertigte Assets zurückgreifen, um den Fokus stärker auf die Logik zu legen. Für die Umgebung wurde dann auch aus Zeitgründen auf besagte vorgefertigte Assets zurückgegriffen.

5.2. Ausblick

Die vorliegende Simulation bietet eine solide Basis, die durch weitere Features deutlich erweitert werden kann. Folgende Punkte wären sinnvolle nächste Schritte für Version 2.0:

- Editor / Bau-Modus: Es könnte ein Editor implementiert werden, der es dem Nutzer erlaubt, die Strecke modular aus vorgefertigten Bauteilen selbst zusammenzusetzen.
- Erweiterte Hindernis-Logik: Die aktuellen Hindernisse könnten durch dynamische Elemente ergänzt werden (z. B. rotierende Räder, bewegliche Wände), die ebenfalls durch Zufallsfunktionen gesteuert werden.
- Grafische Aufwertung: Die 3D-Modelle der Strecke und der Umgebung könnten überarbeitet werden. Auch zusätzliche Umwelteinflüsse (Regen, der die Reibung verändert) wären denkbar.
- Skalierbarkeit & Multiplayer: Aktuell ist die Simulation auf eine lokale Instanz beschränkt. Ein spannender Ausbau wäre die Netzwerkfähigkeit. Zudem sollte die Anzahl der simulierten Murmeln dynamisch zur Laufzeit wählbar sein.



Literaturverzeichnis

- Asanger, Andreas (2024): Blender. 1. Auflage, Rheinwerk Verlag
- Epic Games (2024): Epic C++ Coding Standard for Unreal Engine. Online verfügbar unter: <https://dev.epicgames.com/documentation/en-us/unreal-engine/epic-cplusplus-coding-standard-for-unreal-engine>.
- Epic Games (2024): Unreal Engine 5.4 Documentation. Online verfügbar unter: <https://dev.epicgames.com/documentation/en-us/unreal-engine>.
- International Organization for Standardization (2020): ISO 9241-110:2020 – Ergonomie der Mensch-System-Interaktion.
- Kalista, Heiko (2016): C++ für Spieleprogrammierer. 1. Auflage, Carl Hanser Verlag
- Richartz, Jonas (2017): Spiele entwickeln mit Unreal Engine 4 . 1. Auflage, Carl Hanser Verlag.

Anhang

Tabellen:

Risiko	Eintrittswahrscheinlichkeit	Auswirkung	Maßnahmen
Performance auf Zielhardware	Hoch	Hoch	Kein Lumen/Nanite, Static Lighting, Physik in C++
Fehler beim Packaging (Build)	Mittel	Hoch	Frühzeitige Test-Builds, Regelmäßige Log-Prüfung
Technische Komplexität	Mittel	Mittel	Offizielle Doku nutzen, Blueprint-Fallback
Kollisionsfehler (Tunneling)	Mittel	Hoch	CCD aktivieren, Sub-Stepping erhöhen
Modellierungsfehler (Physik)	Mittel	Mittel	Plausibilitätschecks, Manueller Soll-Ist-Vergleich
Falsche Zufallsverteilung	Niedrig	Mittel	Testreihen visualisieren, Histogramm-Validierung
Scope Creep	Hoch	Niedrig	Einhaltung "Fixed Scope", Priorisierung

Tabelle 3: Risikoauflistung

Kategorie	Asset-Bezeichnung / Beschreibung	Quelle / Plattform
Umgebung	Old Brick House	https://www.fab.com/de/listings/d89c5c69-2737-45cb-9964-27ced2d25024
Plugin	Mini-Graph – God HUD	https://www.fab.com/de/listings/7daa3500-c2e8-4e4f-880f-a8bc0a259025
Requisite	Laptop 3D Model	https://www.fab.com/de/listings/93959379-a1e5-4b07-8484-d9bd44655dfd
Requisite	Ventilator (Fan)	https://fab.com/s/c24cc91c4f0f
Requisite	Hund (Dog)	https://fab.com/s/78d85d73153f

Tabelle 4: Alle genutzten vorgefertigten Assets

Kriterium	Gew.	Unreal Engine	Unity	Godot	Begründung
Physik-Engine	4	3 (Chaos Physics)	2 (PhysX)	1 (Basis)	Unreal bietet mit Chaos Physics die robusteste integrierte Lösung.
GUI-Framework	3	3 (UMG)	2 (UI Toolkit)	2 (Control Nodes)	Das Unreal Motion Graphics Framework bietet einen visuellen Designer.
Programmiersprache	2	3 (C++ / BP)	1 (C#)	2 (GDScript)	Der Hybrid-Ansatz vereint maximale Performance mit schneller Iteration.
Dokumentation & Community	2	3 (Sehr gut)	3 (Sehr gut)	2 (Gut)	Exzellente offizielle Dokumentation und riesige Community.
Einarbeitungszeit	1	3 (Vorwissen)	2 (Basis)	1 (Keine)	Vorkenntnisse sind vorhanden.
Gesamtnutzwert		36	26	19	

Tabelle 5: Nutzwertanalyse Engine

Kriterium	Gew.	JetBrains Rider	Visual Studio	VS Code	Begründung
UE-Integration	4	3 (Tief)	2 (Gut)	1 (Basis)	Zeigt Blueprint-Bindung direkt im Code.
Performance	3	3 (Schnell)	2 (Mittel)	3 (Schnell)	Deutlich schnellere Indizierung als Visual Studio.
Kosten	2	1 (Teuer*)	3 (Kostenlos)	3 (Kostenlos)	*Nachteil wird durch kostenlose Bildungslizenz ausgeglichen.
Plugins	1	2 (Spezifisch)	2 (Viel)	3 (Riesig)	Rider benötigt kaum Plugins, da UE-Tools integriert sind.
Gesamtnutzwert		25	23	19	

Tabelle 6: Nutzwertanalyse IDE

Kriterium	Gew.	Blender	Maya	Cinema 4D	Begründung
Kompatibilität	4	3 (Sehr gut)	3 (Standard)	2 (Gut)	Blender bietet eine direkte Schnittstelle.
Lizenzkosten	4	3 (Kostenlos)	1 (Teuer)	1 (Teuer)	Blender ist Open Source.
Funktionsumfang	2	3 (Umfangreich)	3 (Profi)	3 (Profi)	Alle Tools bieten mehr als für das Projekt benötigt wird.
Gesamtnutzwert		30	22	18	

Tabelle 7: Nutzwertanalyse Modellierungssoftware

ID	Testfall / Aktion	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
1	Murmel auf der Werkbank anklicken	Murmel wird angehoben; Konfigurationsmenü öffnet sich mit korrekten Werten.	Wie erwartet.	OK
2	Selektierte Murmel erneut anklicken	Murmel senkt sich ab; Menü schließt sich.	Wie erwartet.	OK
3	Andere Murmel	Alte Murmel sinkt, neue hebt	Wie erwartet.	OK

	anklicken (während eine selektiert ist)	sich; Menü-Werte aktualisieren sich auf die neue Auswahl.		
4	Murmel auf der Startbahn anklicken (nicht Werkbank)	Keine Interaktion (Murmeln sind dort nicht konfigurierbar).	Keine Interaktion.	OK
5	"Zufall"-Button auf Werkbank (keine Selektion)	Alle Murmeln erhalten Zufallswerte; Übertragung an Startposition; Positionen bleiben erhalten.	Wie erwartet.	OK
6	"Zufall"-Button auf Werkbank (mit Selektion)	Alle Murmeln erhalten Zufallswerte; Menü der selektierten Murmel aktualisiert sich live.	Wie erwartet.	OK
7	"Standard"-Button auf Werkbank (keine Selektion)	Alle Murmeln werden auf Standardwerte zurückgesetzt.	Wie erwartet.	OK
8	"Standard"-Button auf Werkbank (mit Selektion)	Alle Murmeln Reset auf Standard; Menü aktualisiert sich live.	Wie erwartet.	OK
9	Maus-Hover über Text im Konfigurator	Tooltips mit Erklärungen erscheinen.	Wie erwartet.	OK
10	Slider im Menü bewegen	Werte ändern sich innerhalb der Grenzen und in definierten Schrittgrößen.	Wie erwartet.	OK
11	Startposition tauschen (Slot-Wechsel)	Murmeln tauschen physisch und im Datenmodell die Plätze; Menü zeigt korrekte neue Position.	Wie erwartet.	OK
12	"Zufall"-Button im Einzel-Menü drücken	Zufallswerte (innerhalb Slider-Grenzen) nur für diese Murmel; UI aktualisiert sich.	Wie erwartet.	OK
13	"Standard"-Button im Einzel-Menü drücken	Werte dieser Murmel werden auf Default zurückgesetzt; UI aktualisiert sich.	Wie erwartet.	OK
14	Schließen-Button (X) im Menü klicken	Menü schließt sich; Murmel senkt sich ab.	Wie erwartet.	OK
15	Reset-Button (HUD oben links) klicken	Simulation setzt zurück; sonst keine Aktion.	Wie erwartet.	OK
16	Schließen-Button (HUD oben rechts)	Anwendung beendet sich.	Wie erwartet.	OK
17	Option-Button (HUD oben rechts)	Hauptmenü (Overlay) öffnet sich.	Wie erwartet.	OK
18	Checkbox "Hindernisse" umschalten	Hindernisse verschwinden bzw. werden neu generiert.	Wie erwartet.	OK
19	Spinboxen (Maus-Interaktion)	Werteänderung innerhalb der definierten Min/Max-Grenzen.	Wie erwartet.	OK
20	Spinboxen (Tastatur-)	Ungültige Werte/Buchstaben	Wie erwartet.	OK

	Eingabe)	werden ignoriert; Werte außerhalb der Grenzen werden auf Min/Max korrigiert (Clamping).		
21	Checkbox "Hund" umschalten	Hund wechselt Animation (Hinlegen <-> Aufstehen).	Wie erwartet.	OK
22	Checkbox "Hund" schnell umschalten	State Machine bleibt stabil; Hund verbleibt im zuletzt gewählten Zustand.	Wie erwartet.	OK
23	Checkbox "Ventilator" umschalten	Rotorblatt-Animation startet/stoppt.	Wie erwartet.	OK
24	Auflösung ändern (Button)	Auflösung ändert sich; Button-Status (Selected) aktualisiert sich.	Wie erwartet.	OK
25	Sprache "Englisch" wählen	Sprache der UI wechselt auf Englisch.	Fehler: Sprache wechselt nicht.	Nein
26	"Standard"-Button im Hauptmenü	Nur Zufallselemente/Spinboxen werden zurückgesetzt.	Wie erwartet.	OK
27	"Schließen" im Hauptmenü	Menü schließt sich; Rückkehr zur Ansicht.	Wie erwartet.	OK
28	Options-Button drücken (während Menü offen)	Menü schließt sich (Toggle-Funktion).	Wie erwartet.	OK
29	Hindernis-Parameter ändern (aktiviert)	Hindernisse werden bei Wertänderung sofort neu generiert.	Wie erwartet.	OK
30	Hindernis-Parameter ändern (deaktiviert)	Keine Änderung sichtbar (Performance sparen).	Wie erwartet.	OK
31	"Neu Generieren"-Button (Hindernisse an)	Neue Zufallsgenerierung der Strecke.	Wie erwartet.	OK
32	"Neu Generieren"-Button (Hindernisse aus)	Keine Aktion.	Wie erwartet.	OK
33	Start-Button drücken	Rennen startet; Murmeln rollen; Zeitmessung (HUD) erscheint.	Wie erwartet.	OK
34	Reset-Button während des Rennens	Abbruch; Kamera zurück zur Werkbank; Timer verschwindet; Murmeln despawnen.	Wie erwartet.	OK
35	Pause/Play während des Rennens	Physik friert ein/läuft weiter; Timer stoppt/läuft weiter.	Wie erwartet.	OK
36	Reset drücken während Pause	Pause wird aufgehoben; Reset wird ausgeführt (Kamerafahrt zur Config).	Wie erwartet.	OK
37	Geschwindigkeit ändern (Time Dilation)	Simulationsgeschwindigkeit und Timer beschleunigen/verlangsamen sich synchron.	Wie erwartet.	OK

38	Abbruch des Rennens	Kamera wechselt zum Analyse-Screen; nur Murmeln im Ziel werden gewertet.	Wie erwartet.	OK
39	Ziel: Alle Murmeln kommen an	Wechsel zum Analyse-Screen; alle Platzierungen in Tabelle; Graphen vollständig.	Wie erwartet.	OK
40	Ziel: Teilsieg (einige eliminiert)	Wechsel zum Analyse-Screen; eliminierte Murmeln haben keine Platzierung.	Wie erwartet.	OK
41	Ziel: Keine Murmel kommt an	Wechsel zum Analyse-Screen; leere Tabelle/Graphen.	Wie erwartet.	OK
42	Analyse: Checkboxes Murmeln	Graphen der jeweiligen Murmel werden ein-/ausgeblendet; Skalierung passt sich an.	Wie erwartet.	OK
43	Analyse: Interaktion mit Graph	Keine Interaktion (nur Anzeige); Mauszeiger ändert sich nicht.	Wie erwartet.	OK
44	Analyse: Slider bewegen (Zeitachse)	Graphen aktualisieren/glätten sich dynamisch.	Wie erwartet.	OK
45	Analyse: Reset-Button	Rückkehr zur Werkbank; Standard-Kamera/Bildschirmschoner aktiv.	Wie erwartet.	OK
46	Analyse: Datenkonsistenz	Anzeige zeigt nur Daten des letzten Rennens (keine alten Datenreste).	Wie erwartet.	OK
47	Physik-Test: Ohne Einflüsse	Keine seitlichen Kräfte; keine Stöße; keine Kollisionen (gerade Bahn).	Wie erwartet.	OK
48	Renn-Kamera: Tasten 1-5 / Tab	Kamera fokussiert entsprechende Murmel bzw. wechselt zur nächsten aktiven.	Wie erwartet.	OK
49	Renn-Kamera: Ziel/Crash bei Fokus	Kamera wechselt automatisch zur nächsten noch aktiven Murmel.	Wie erwartet.	OK
50	Performance / Stresstest	1 Stunde Laufzeit / 50 Rennen in Folge.	Stabil; keine Frame-Einbrüche (Lags); kein Absturz.	OK

Tabelle 8: Testprotokoll

Kategorie	Anforderung (Soll)	Status (Ist)	Anmerkung / Umsetzung
Funktion	Zeitabhängige Simulation	Erfüllt	Simulation läuft in Echtzeit/angepasster Zeit.
Mathematik	Mind. 3 Zufallsverteilungen (z.B. Normal, Gleich, Expo)	Erfüllt	Implementiert: ADog (Expo), AFan (Normal), Obstacle (Gleich).
Input	Mindestens 7 Eingabeparameter	Erfüllt	User kann >7 Werte im UI konfigurieren.
Steuerung	Simulationsgeschwindigkeit in mind. 3 Stufen	Erfüllt	Geschwindigkeit ist variabel einstellbar (1,2,5-fach).
Visuelles	Visuelle Darstellung & Animation	Erfüllt	UI-Visualisierung mittels Graph und tabellarisch vorhanden. Murmel rollt visuell.
System	Windowsanwendung / Ausführbare Datei (.exe) auf Schulrechnern	Erfüllt	Build als .exe erstellt und getestet auf den Schulrechnern.
Qualität	Clean-Code-Kriterien	Erfüllt	Code-Struktur und Benennungsregeln eingehalten. Es wurde sich an die offizielle Unreal Dokumentation gehalten und Funktionen auch mit Docstrings versehen.
UX / UI	Standard ISO 9241-110 & benutzerfreundliche Animation	Erfüllt	UI ist intuitiv gestaltet, Feedback durch Animationen.
Doku	Benutzerhandbuch digital bereitgestellt	Erfüllt	PDF/Online-Hilfe liegt bei.

Doku	Umfangreiche Dokumentation nach IHK-Standard	Erfüllt	Mindestens 6 und maximal 12 Seiten Fließtext um die Entscheidungen zu begründen.
------	--	---------	--

Tabelle 9: Soll-Ist-Vergleich

Bilder:

Startdatum: 06.09.2025 | Enddatum: 06.02.2026
Dauer: 23 Wochen | Geplante Stunden: 160h

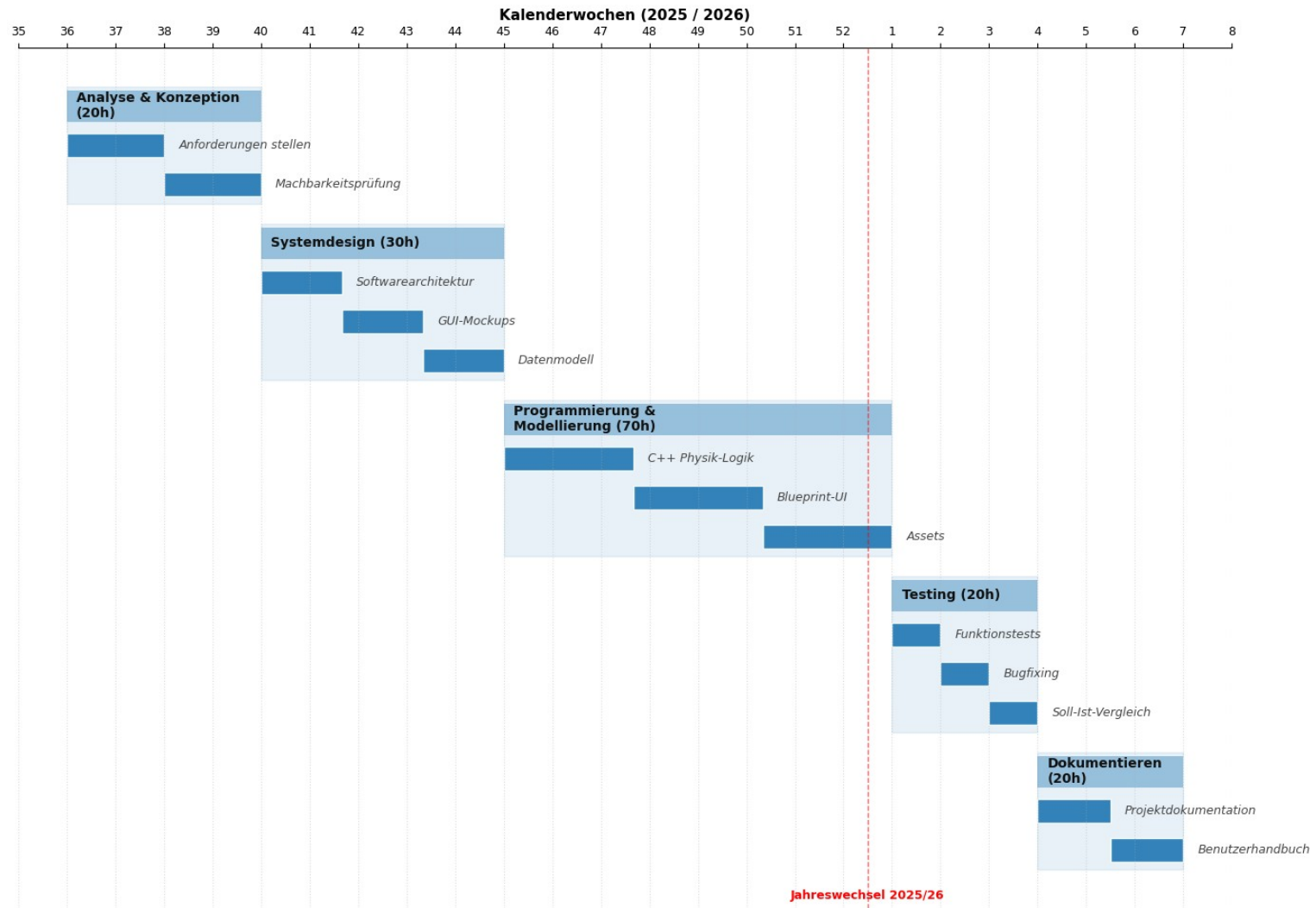


Schaubild 1: Gantt-Diagramm zur Zeitplanung

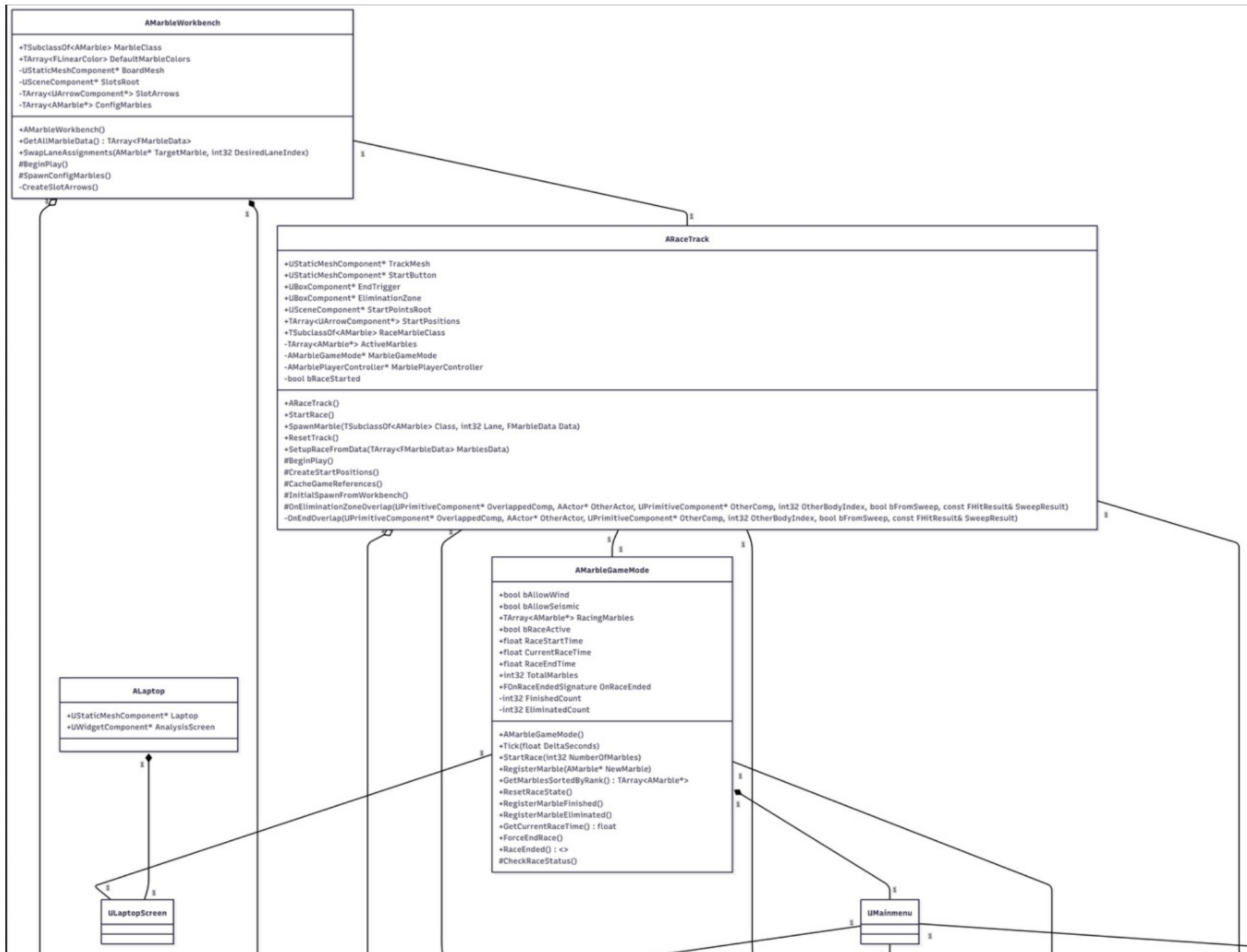


Schaubild 3: Klassendiagramm (Teil 1 mit Attributen und Funktionen)

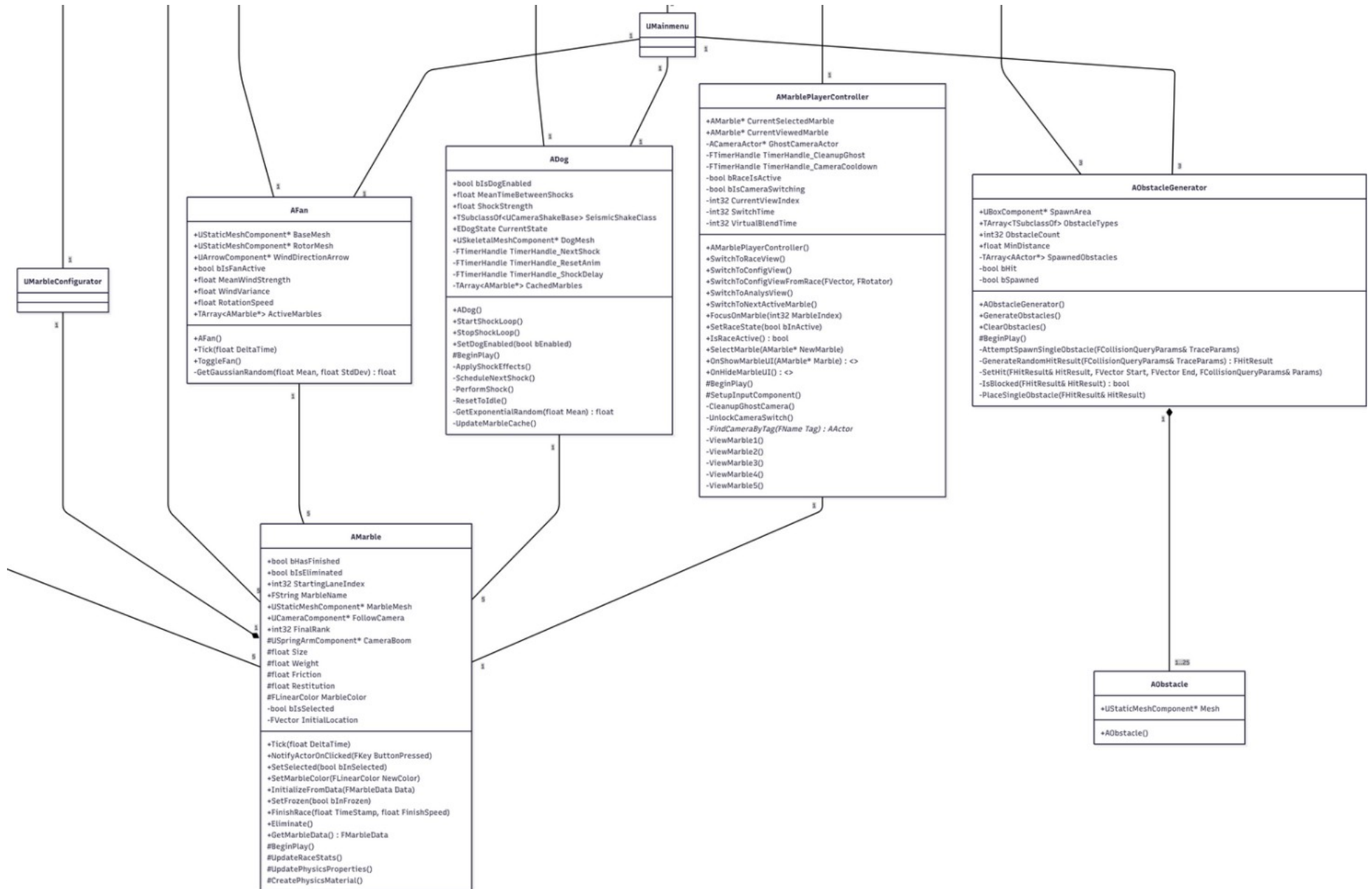


Schaubild 4: Klassendiagramm (Teil 2 mit Attributen und Funktionen)

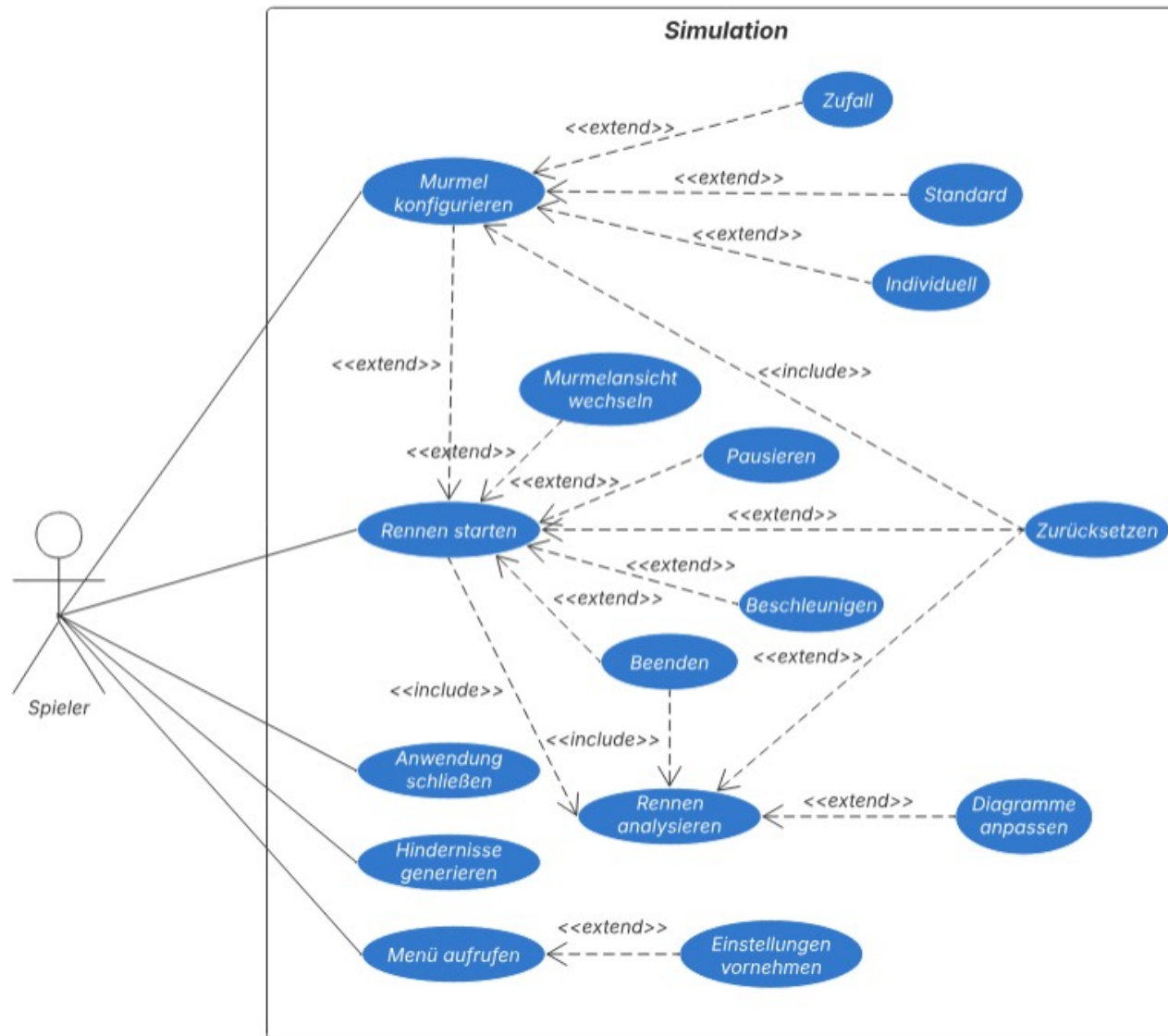


Schaubild 5: Use-Case-Diagramm der Simulation